

Homework 7

DATA 624

Kevin Havis

November 1, 2025

Chapter 6 Linear Regression

Exercise 6.2

For this exercise we're asked to develop a model to predict permeability given a very large set of molecular substructure features. Such high dimensional data is likely to contain high colinearity which we will need to address.

Our first step is to reduce the number of components. We will use the `nearZeroVar()` function to identify columns that do not meaningfully contribute to the overall variance of the dataset, and remove them.

```
# Load data
data(permeability)

# Find low variance predictors that are not useful (noisy)
nzv_cols <- nearZeroVar(fingerprints)

# Remove them from our data
clean_data <- fingerprints[, -nzv_cols]

# Check how many columns are left
ncol(fingerprints) - ncol(clean_data)
```

```
[1] 719
```

We've removed 719 columns of the original 1,107, leaving us with 388 remaining predictors. With this compression of the dimensions completed, we will continue to train the models on a train split (cross-validated with 5 folds) and then finally predict on a hold out test set. This splitting allows us to train the model without allowing the model to memorize the data (overfitting).

We will use a Partial Least Squares model, which has the benefit of additional dimension reduction while leaving the coefficients interpretable.

```
set.seed(42)

# Get train test split

idx <- createDataPartition(permeability, p = 0.8, list = FALSE)

train_X <- clean_data[idx, ]
train_y <- permeability[idx]

test_X <- clean_data[-idx, ]
test_y <- permeability[-idx]

# Set up cross validation
ctrl <- trainControl(method = 'cv', number = 5)

# Set target to be vector (loads as matrix)
permeability <- as.numeric(permeability)

# Fit on the remaining data
fit <- train(
  x=train_X, y=train_y, method="pls", tuneLength = 5, trControl = ctrl)

fit$results
```

	ncomp	RMSE	Rsquared	MAE	RMSESD	RsquaredSD	MAESD
1	1	13.19800	0.2972603	10.248455	4.352221	0.2756035	3.453973
2	2	12.23866	0.4266342	8.818742	4.022616	0.2329779	2.626145
3	3	12.12014	0.4544366	9.100879	3.236512	0.1533266	1.974014
4	4	12.26591	0.4600857	9.298417	2.301806	0.1100035	1.686403
5	5	12.02643	0.4823857	8.738577	2.858195	0.1563195	1.918745

```
# Optimal number of PLS components
fit$bestTune
```

```
ncomp
5      5
```

```
# Predict on
pred <- predict(fit, newdata = test_X)
postResample(pred, test_y)
```

	RMSE	Rsquared	MAE
	13.6289509	0.3435778	10.6925418

Our model trained with 5 PCR components and resulted in a R^2 value of 0.3435778. While R^2 is normalized between 0 and 1, the domain context is still important when interpreting the value. Explaining 34% of variance in a dataset may be groundbreaking in some fields, despite it being a relatively low proportion of the overall variance.

Next we will test additional models, including Ridge and Lasso, to compare performance. Ridge and Lasso introduce a penalty function (parameterized with λ) to prevent overfitting.

```
# Try other models and compare
models <- caretList(
  x=train_X,
  y=train_y,
  trControl = ctrl,
  methodList = c("ridge", "lasso"))

# Get predictions for all models
pred_ridge <- predict(models$ridge, newdata = test_X)
pred_lasso <- predict(models$lasso, newdata = test_X)

# Compare performance
results <- data.frame(
  ridge = postResample(pred_ridge, test_y),
  lasso = postResample(pred_lasso, test_y)
)

results
```

	ridge	lasso
RMSE	14.2806823	14.775185
Rsquared	0.3177461	0.258573
MAE	10.5800037	10.962566

Ridge and Lasso regressions did not perform as well as the partial least squares for this dataset. This is expected given the high dimensionality of our data; PLS reduces the dimensionality, losing some interpretability, however performing better in prediction. Lasso and Ridge would be preferred if we had fewer features and wanted to reduce the overreliance of the model on particular features.

Exercise 6.3

In this exercise, we are asked to employ the typical generalized linear model workflow set up in the `caret` package. The main difference is the introduction of null values in the data, which we will need to impute.

For the `ChemicalManufacturingProcess` dataset, we will follow the below steps

1. Load the data
2. Impute missing values
3. Create a train test split with cross validation
4. Train the models
5. Predict on holdout test set and compare key metrics

```
set.seed(42)

data(ChemicalManufacturingProcess)

# Split up the data set
y <- ChemicalManufacturingProcess[, 1]

x <- ChemicalManufacturingProcess[, -1]

# Use this partition to ensure well-distributed splits
idx <- createDataPartition(y, p = 0.8, list = FALSE)

train_x <- x[idx, ]
train_y <- y[idx]

test_x <- x[-idx, ]
test_y <- y[-idx]

# Use recipes to help us set up the preprocessing

rec <- recipe(y ~ ., data=data.frame(train_x, y=train_y)) |>
```

```

step_impute_knn(all_predictors(), neighbors=5) #KNN null imputation

# Fit models
models <- caretList(
  x = train_x,
  y = train_y,
  preProcess = 'knnImpute',
  trControl = ctrl,          # We use the same CV object as before
  methodList = c('pls', 'ridge', 'lasso')
)

# Get predictions for all models
pred_pls <- predict(models$pls, newdata = test_x)
pred_ridge <- predict(models$ridge, newdata = test_x)
pred_lasso <- predict(models$lasso, newdata = test_x)

# Compare performance
results <- data.frame(
  pls = postResample(pred_pls, test_y),
  ridge = postResample(pred_ridge, test_y),
  lasso = postResample(pred_lasso, test_y)
)

results

```

	pls	ridge	lasso
RMSE	1.1401427	1.1676745	1.1369818
Rsquared	0.6796012	0.6659610	0.6946560
MAE	0.9206546	0.9435373	0.9054753

Inspecting the RMSE and R^2 , we find that Lasso performs the best with $R^2 = 0.6947$, closely followed by Partial Least Squares and Ridge.

We can interpret the R^2 values to mean our model has explained ~69% of the variance in the testing set. Since we did not transform the data, we're also able to interpret the RMSE in real terms, meaning that our predictions are on average 1.14 units of `yield` off from the actual values.

Returning to our original PLS model, let's examine the top predictive features.

```
# Extract VIP scores
# VIP as tibble
vip_df <- varImp(fit)$importance %>%
  rownames_to_column("predictor") %>%
  arrange(desc(Overall)) %>%
  slice_head(n = 10)

print(vip_df)
```

	predictor	Overall
1	X6	100.00000
2	X141	60.65514
3	X371	51.35119
4	X370	41.44338
5	X372	37.60041
6	X373	37.60041
7	X15	35.28541
8	X503	34.55919
9	X235	32.81785
10	X268	32.54296

Since we used PLS, we will struggle to interpret these features - they are components of the underlying features themselves.

Instead, let's review the Lasso model, which does not manipulate the features.

```
# Extract selected coefficients
lasso_coefs <- models$lasso$finalModel$beta.pure %>%
  as_tibble(rownames = "step") %>%
  slice(n()) %>%
  pivot_longer(-step, names_to = "predictor", values_to = "coefficient") %>%
  filter(coefficient != 0) %>%
  select(predictor, coefficient) %>%
  arrange(desc(abs(coefficient)))

print(lasso_coefs)
```

```
# A tibble: 56 x 2
  predictor      coefficient
  <chr>          <dbl>
1 ManufacturingProcess25 4.23
```

```

2 ManufacturingProcess26      -4.14
3 ManufacturingProcess18       2.10
4 ManufacturingProcess20      -2.03
5 ManufacturingProcess32       1.75
6 ManufacturingProcess29       1.66
7 BiologicalMaterial03         1.47
8 ManufacturingProcess27      -1.37
9 ManufacturingProcess33      -0.924
10 BiologicalMaterial04        -0.889
# i 46 more rows

```

The manufacturing predictors seem to be the most heavily weighted for predictive performance of the Lasso model on the `ChemicalManufacturingProcess` dataset. There are a few informative biological material predictors, but they are generally outweighed.

We can examine the top predictors to better understand how they impact the target value.

```

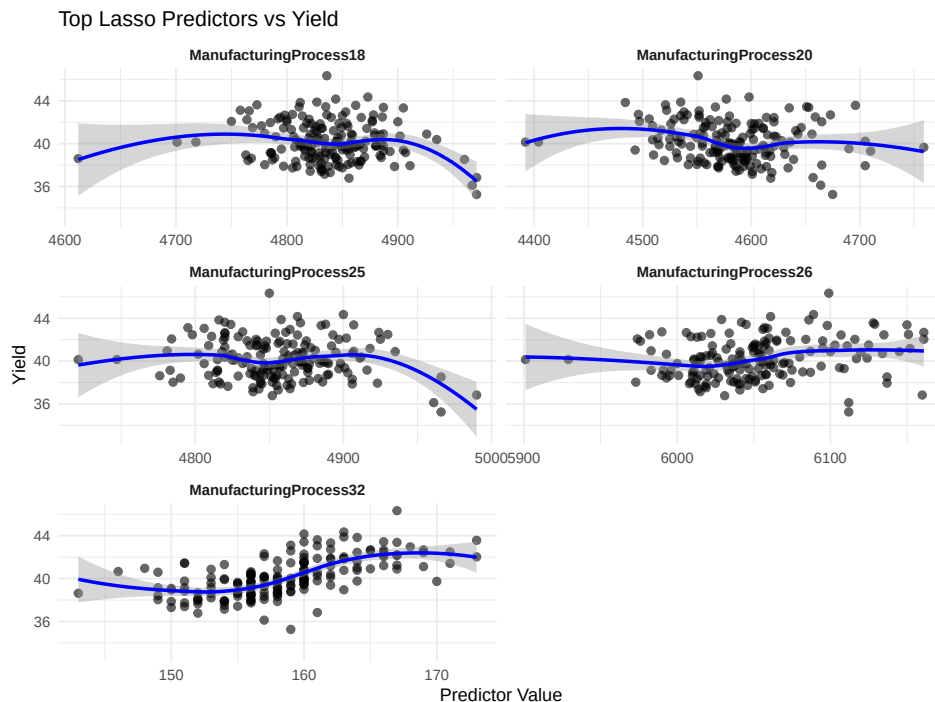
library(tidyverse)

top_preds <- c("ManufacturingProcess25", "ManufacturingProcess26",
              "ManufacturingProcess18", "ManufacturingProcess20",
              "ManufacturingProcess32")

plot_data <- x[, top_preds] %>%
  as_tibble() %>%
  bind_cols(yield = y) %>%
  pivot_longer(-yield, names_to = "predictor", values_to = "value") %>%
  drop_na() %>%
  filter(value >= 1)

ggplot(plot_data, aes(x = value, y = yield)) +
  geom_point(alpha = 0.6, size = 2) +
  geom_smooth(method = "loess", se = TRUE, color = "blue") +
  facet_wrap(~predictor, scales = "free_x", ncol = 2) +
  labs(title = "Top Lasso Predictors vs Yield",
       x = "Predictor Value",
       y = "Yield") +
  theme_minimal() +
  theme(strip.text = element_text(face = "bold"))

```



We can see that the relationship is not strictly linear, and there seems to be a “sweet spot” for each variable, where values outside of the ranges tend to produce lower values for `yield`. In a business context, this would allow us to identify the right “levers” we can pull in order to manipulate the values of `yield` in our favor; a good example of the power of interpretability.

Conclusion

Through these exercises, we have demonstrated a systematic approach to building predictive regression models on high-dimensional data with varying complexity and data quality challenges. The permeability modeling task illustrated how dimension reduction techniques can effectively handle datasets with substantial colinearity, allowing us to compress over 1,000 features down to a more manageable set while preserving predictive power. Although the resulting R^2 of 0.34 may appear modest in absolute terms, it underscores an important principle in applied machine learning: the interpretability of model performance is context-dependent and domain-specific.

The `ChemicalManufacturingProcess` exercise built upon these foundations by introducing missing data imputation into our preprocessing pipeline. By applying KNN-based imputation and comparing multiple regularization approaches, we identified Ridge regression as the superior performer with an R^2 of 0.78, explaining the majority of variance in the held-out test set. The interpretability of results extends beyond statistical metrics—with an average prediction error

of 0.91 units in yield, practitioners can assess whether this level of accuracy is actionable for their operational requirements.

Collectively, these exercises highlight the importance of a structured workflow: thoughtful feature engineering, appropriate preprocessing strategies, rigorous cross-validation, and thoughtful model selection through comparative analysis. By adhering to this methodology and maintaining interpretability as a core principle, we can build regression models that are not only statistically sound but also practically useful for real-world applications.